

Análise Comparativa de Algoritmos de Busca em Strings

Relatório Técnico

Naive · Rabin-Karp · KMP · Boyer-Moore

Rafael Pavesi dos Passos

Lucas Warmiling

2026

1. Introdução

A busca de padrões em strings (pattern matching) é um dos problemas fundamentais da Ciência da Computação, com aplicações em editores de texto, bioinformática, motores de busca, segurança de redes e análise de log. O problema consiste em, dado um texto T de comprimento n e um padrão P de comprimento m , encontrar todas as ocorrências de P em T de forma eficiente.

Este relatório documenta a implementação, visualização passo a passo e comparação de desempenho de quatro algoritmos clássicos: Naive, Rabin-Karp, Knuth-Morris-Pratt (KMP) e Boyer-Moore. A aplicação desenvolvida utiliza o padrão de projeto Strategy para encapsular cada algoritmo como uma estratégia intercambiável, permitindo execução normal e passo a passo com visualização interativa.

1.1 Padrão de Projeto Strategy

O padrão Strategy define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. No contexto desta aplicação, a interface `SearchStrategy` declara os métodos `search()` e `getSteps()`, e cada algoritmo implementa essa interface:

Interface / Classe	Papel	Complexidade
<code>SearchStrategy</code>	Interface base (abstrata)	—
<code>NaiveSearch</code>	Força bruta, sem pré-processamento	$O(n \cdot m)$
<code>RabinKarpSearch</code>	Hashing com rolling hash	$O(n+m)$ médio
<code>KMPSearch</code>	Tabela LPS, sem retrocesso em T	$O(n+m)$
<code>BoyerMooreSearch</code>	Heurística Bad Character	$O(n/m)$ melhor
<code>SearchContext</code>	Contexto que usa a estratégia	—

Tabela 1 — Arquitetura Strategy implementada

2. Algoritmos Implementados

2.1 Busca Naive (Força Bruta)

O algoritmo mais simples de busca de padrão. Para cada posição i no texto (de 0 a $n-m$), alinha o padrão e compara caractere a caractere da esquerda para a direita. Em caso de mismatch, avança i em 1 e reinicia a comparação do padrão.

Pseudocódigo:

```
para i de 0 até n-m:
    j = 0
    enquanto j < m e T[i+j] == P[j]:
        j++
    se j == m: ocorrência em i
```

Análise de Complexidade:

- Melhor caso $O(n)$: o padrão não ocorre e o primeiro caractere não aparece no texto.
- Caso médio $O(n \cdot m)$: comparações repetidas em cada posição.
- Pior caso $O(n \cdot m)$: texto e padrão compostos pelo mesmo caractere — m comparações em cada posição.
- Espaço $O(1)$: nenhuma estrutura auxiliar.

Quando usar: adequado para textos e padrões pequenos ou quando a implementação simples é preferível. Ineficiente para grandes volumes de dados.

2.2 Rabin-Karp

Utiliza hashing para comparar o padrão com cada janela de texto. O hash do padrão é calculado uma vez; para cada posição, o hash da janela atual é mantido com o rolling hash, que remove a contribuição do caractere que sai e adiciona o novo em $O(1)$. Quando os hashes coincidem, faz-se verificação caractere a caractere para confirmar, evitando falsos positivos.

Hash Polinomial:

```
h(s) = (s[0] · d^(m-1) + s[1] · d^(m-2) + ... + s[m-1]) mod q
Rolling: t[i+1] = (d · (t[i] - T[i] · h) + T[i+m]) mod q
Onde d=256 (tamanho do alfabeto) e q é um número primo (ex: 101).
```

Análise de Complexidade:

- Melhor / médio $O(n+m)$: poucas colisões; cada posição é descartada pelo hash em $O(1)$.
- Pior caso $O(n \cdot m)$: muitas colisões forçam verificação caractere a caractere em cada posição.
- Espaço $O(1)$: apenas variáveis de hash.

Quando usar: excelente para busca de múltiplos padrões simultaneamente. Muito usado em detecção de plágios e análise de DNA.

2.3 Knuth-Morris-Pratt (KMP)

O KMP elimina comparações redundantes ao aproveitar informações sobre o padrão já comparadas. Pré-processa o padrão para construir a tabela LPS (Longest Proper Prefix which is also Suffix), que indica quanto do padrão pode ser reutilizado após um mismatch.

Construção da tabela LPS:

```
LPS[0] = 0 (sempre)
para i de 1 até m-1:
    se P[i] == P[len]: LPS[i] = ++len
    senão se len > 0: len = LPS[len-1] // sem avançar i
    senão: LPS[i] = 0
```

Exemplo para $P = \text{'AAACAAAA'}$: $LPS = [0, 1, 2, 0, 1, 2, 3, 3]$. Em caso de mismatch em j , em vez de recomeçar do início, recua j para $LPS[j-1]$, aproveitando a correspondência anterior.

Análise de Complexidade:

- Pré-processamento $O(m)$: construção da tabela LPS.
- Busca $O(n)$: o índice i nunca retrocede — cada caractere do texto é visitado no máximo uma vez.
- Total $O(n+m)$: garantido no melhor e no pior caso.

- Espaço $O(m)$: para armazenar a tabela LPS.

Quando usar: ideal para streaming (texto lido uma vez) e padrões com estrutura repetitiva. Garante $O(n+m)$ em todos os cenários.

2.4 Boyer-Moore

O mais eficiente na prática para textos grandes com alfabeto rico. Compara o padrão da direita para a esquerda e usa a heurística Bad Character para calcular saltos maiores que 1 após um mismatch, pulando grandes porções do texto.

Tabela Bad Character:

```
para cada caractere c em P:  
    badChar[c] = última posição de c em P  
Se c não aparece em P: badChar[c] = -1  
  
shift = max(1, j - badChar[T[s+j]])
```

Análise de Complexidade:

- Melhor caso $O(n/m)$: o padrão salta m posições a cada verificação.
- Caso médio $O(n/m)$: em textos com alfabeto grande, os saltos são frequentemente próximos de m.
- Pior caso $O(n \cdot m)$: alfabeto muito pequeno com muitas correspondências parciais.
- Espaço $O(\sigma)$: para a tabela Bad Character (σ = tamanho do alfabeto).

Quando usar: o melhor algoritmo para textos longos e alfabeto grande. É o algoritmo usado internamente em grep e ferramentas profissionais de busca.

3. Análise Comparativa de Desempenho

3.1 Tabela de Complexidades

Algoritmo	Pré-proc.	Melhor Caso	Caso Médio	Pior Caso	Espaço
Naive	—	$O(n)$	$O(n \cdot m)$	$O(n \cdot m)$	$O(1)$
Rabin-Karp	$O(m)$	$O(n+m)$	$O(n+m)$	$O(n \cdot m)$	$O(1)$
KMP	$O(m)$	$O(n)$	$O(n+m)$	$O(n+m)$	$O(m)$
Boyer-Moore	$O(m+\sigma)$	$O(n/m)$	$O(n/m)$	$O(n \cdot m)$	$O(\sigma)$

Tabela 2 — Complexidades teóricas (σ = tamanho do alfabeto, m = padrão, n = texto)

3.2 Cenários de Teste Representativos

Os resultados abaixo são representativos da execução dos algoritmos em diferentes cenários. Os valores de comparações e tempo variam com o hardware e a implementação.

Cenário	Algoritmo	n	m	Comparações	Ocorrências	Observação
Texto curto, palavra comum	Naive	100	3	~150	5	Poucos saltos possíveis
	KMP	100	3	~100	5	Sem retrocesso em T
	Boyer-Moore	100	3	~60	5	Melhor p/ alfabeto rico
Texto longo, padrão ausente	Naive	10000	10	~10000	0	1 comp. por posição
	Boyer-Moore	10000	10	~1000	0	Salta ~m posições
Pior caso (aaaa...)	Naive	1000	10	~9910	991	$n \times m$ comparações
	KMP	1000	10	~1000	991	Apenas $O(n)$

Tabela 3 — Cenários de teste e comparações realizadas (valores aproximados)

4. Quando Cada Algoritmo é Mais Eficiente

Algoritmo	Use quando...	Evite quando...
Naive	n e m pequenos (< 1000); implementação rápida; código didático.	Textos grandes; alta frequência de busca; padrões com prefixos repetidos.
Rabin-Karp	Busca de múltiplos padrões; detecção de plágio; análise de DNA.	Alfabeto muito pequeno (muitas colisões); necessidade de garantia $O(n+m)$.
KMP	Streaming (texto lido uma vez); padrões repetitivos; alfabetos pequenos (DNA, binário).	Textos com alfabeto grande onde Boyer-Moore supera; múltiplos padrões.
Boyer-Moore	Textos em linguagem natural; grep, editores; padrão moderado ($m \geq 5-10$).	Alfabetos muito pequenos (DNA, binário); padrões muito curtos ($m=1,2$).

Tabela 4 — Guia de seleção de algoritmo por contexto

4.1 Regra Prática de Seleção

- Texto natural (artigos, código, logs): Boyer-Moore é o mais rápido na prática.
- Streaming / uma passagem: KMP garante $O(n+m)$ sem lookahead.
- Múltiplos padrões simultâneos: Rabin-Karp ou Aho-Corasick.
- Alfabeto pequeno (DNA: ACGT): KMP supera Boyer-Moore.
- Padrões muito curtos ($m \leq 3$): Naive pode ser competitivo por overhead zero.

5. Arquitetura da Aplicação

A aplicação foi desenvolvida como uma Single Page Application (SPA) em HTML + CSS + JavaScript puro, sem dependências externas, encapsulando toda a lógica em classes que implementam o padrão Strategy.

5.1 Funcionalidades Implementadas

- Upload de um ou múltiplos arquivos .txt com drag & drop.
- Campo de texto editável diretamente na interface.
- Seleção do algoritmo via botões visuais com indicador de complexidade.
- Execução normal: resultado imediato com todos os índices de ocorrência.
- Execução passo a passo: navegação manual e autoplay com velocidade ajustável.
- Visualização dos caracteres do texto e padrão com boxes coloridos por estado (match/mismatch/janela/encontrado).
- Exibição das estruturas auxiliares: tabela LPS (KMP), tabela Bad Character (Boyer-Moore), hashes (Rabin-Karp).
- Log completo de todos os passos com descrição textual.
- Aba de Métricas: comparações, tempo em microssegundos, tamanhos n e m.
- Aba de Comparação: executa todos os 4 algoritmos e exibe gráfico de barras comparativo.

5.2 Padrão Strategy em JavaScript

Como JavaScript não possui interfaces nativas, o padrão é implementado com classes base que lançam erros se não sobrescritas, garantindo o contrato da interface:

```
class SearchStrategy {
  search(text, pattern) { throw new Error('Not implemented'); }
  getSteps(text, pattern) { throw new Error('Not implemented'); }
}

class SearchContext {
```



```
    setStrategy(key) { this.current = this.strategies[key]; }  
    run(text, pattern) { return this.current.search(text,  
pattern); }  
}
```

6. Conclusão

Este trabalho demonstrou a implementação prática e comparativa de quatro algoritmos clássicos de busca em strings, revelando que não existe um algoritmo universalmente superior — cada um tem seu domínio de eficiência:

- O Naive serve como linha de base e é surpreendentemente competitivo para padrões curtos em textos pequenos devido ao seu overhead zero.
- O Rabin-Karp brilha na busca de múltiplos padrões, sendo a base de sistemas anti-plágio e análise de sequências biológicas.
- O KMP garante $O(n+m)$ em qualquer cenário, sendo a escolha certa para streaming e alfabetos pequenos onde Boyer-Moore degrada.
- O Boyer-Moore é o algoritmo mais rápido na prática para texto em linguagem natural, saltando grandes porções do texto — razão pela qual é adotado em grep, vim e outros utilitários profissionais.

O padrão Strategy provou ser uma escolha arquitetural elegante, permitindo trocar algoritmos sem modificar o contexto cliente, facilitando testes comparativos e manutenção do código.

A visualização passo a passo revelou-se fundamental para a compreensão intuitiva do comportamento de cada algoritmo — especialmente a tabela LPS do KMP e os saltos do Boyer-Moore, que são conceitos de difícil absorção apenas pela teoria.